QueryAPIGate HOW-TO GUIDE 14 · ACCESS

Give an external partner access to exactly one query

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=14-give-a-partner-one-query-only

Time: 5 minutes. **You'll end up with:** an API key for someone outside your organisation that runs one saved query and nothing else - no other queries, no ad-hoc SQL, no schema browsing, not even on the same connection.

A `connections` grant is the wrong tool here: it reaches *everything* on a connection - every saved query on it, its schema, and ad-hoc SQL. A `queries` grant names saved queries instead, and reaches only those.

Step 1: The query you want to share

Any saved query works. This guide uses one on a `shop` connection (see Connect a database), next to another query on the same connection that the partner must *not* reach:

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "orders_by_status",
  "sql": "SELECT id, total, status FROM orders WHERE status = :status ORDER BY id",
  "parameters": {"status": {"type": "str", "enum": ["paid", "pending"]}},
  "connection_name": "shop", "description": "Orders with a given status", "publish": true
}'

curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "salary_list", "sql": "SELECT * FROM salaries",
  "connection_name": "shop", "description": "Internal only", "publish": true
}'
```

Declaring `parameters` with rules matters more than usual here: a partner's client is code you don't control, so let the server reject bad input (see Use bound parameters safely).

Step 2: Create the key

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "acme-corp", "connections": [], "queries": ["orders_by_status"]
}'
```

```json
{"name": "acme-corp", "connections": [], "queries": ["orders_by_status"], "collections": [],
 "allow_writes": false, ..., "secret": "sk_VPMGNkB-u-eFXOLORjQ2QDfbANebkVrFlC9imS7XBXg"}
```

`"connections": []` is the important part. Leaving `connections` out entirely means *every* connection (see Set up your first scoped API key) - the opposite of what you want. Copy the `secret` now; it is shown this once.

Step 3: Check what it can and can't do

```shell
curl 'http://127.0.0.1:5000/q/orders_by_status?status=paid' -H 'X-API-Key: <acme-corp secret>'
# [{"id": 1, "total": 42.5, "status": "paid"}, {"id": 2, "total": 87.25, "status": "paid"}]
```

Everything else is refused with `403`, verified:

```shell
# Another saved query on the same connection
curl http://127.0.0.1:5000/q/salary_list -H 'X-API-Key: <acme-corp secret>'
# {"error": "This API key is not permitted to use the connection 'shop'", "code":
"connection_forbidden", ...}

# Ad-hoc SQL
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: <acme-corp secret>' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT * FROM salaries", "connection_name": "shop"}'
# {"error": "This API key is not permitted to use the connection 'shop'", "code":
"connection_forbidden", ...}

# Its own query, pointed at a different connection
curl 'http://127.0.0.1:5000/q/orders_by_status?status=paid&connection_name=examples' -H 'X-API-Key:
<acme-corp secret>'
# {"error": "This API key is not permitted to use the connection 'examples'", "code":
"connection_forbidden", ...}

# The connection's schema
curl http://127.0.0.1:5000/connections/shop/schema -H 'X-API-Key: <acme-corp secret>'
# {"error": "This API key is not permitted to use the connection 'shop'", "code":
"connection_forbidden", ...}
```

What the partner sees of your API

The partner can discover only what they can call. `/openapi.json` (and so `/docs`) lists one saved query for this key - the generic routes, plus `/q/orders_by_status` and nothing else. `GET /catalog` says the same, with the terms attached:

```shell
curl http://127.0.0.1:5000/catalog -H 'X-API-Key: <acme-corp secret>'
```

```json
{"queries": [{"name": "orders_by_status", "version": 1, "description": "Orders with a given status",
              "connection_name": "shop", "parameters": {"status": {"type": "string", "required": true,
              "enum": ["paid", "pending"], ...}}, "cache_ttl": null, "can_write": false}],
 "caller": {"name": "acme-corp", "admin": false, "allow_writes": false, "rate_limit": null, ...}}
```

The SQL text is never included in either. One thing the partner *can* learn: whether a name exists. A query it can't reach answers `403` and names its connection, while a name that doesn't exist answers `404`. Don't put anything sensitive in query or connection names.

When the query changes, the partner's key follows it: a published new version is what `/q/orders_by_status` runs next, with no change to the key.

Variations

- **Several queries:** list them all - `"queries": ["orders_by_status", "order_totals"]`. To share a group that will grow, use a collection grant instead.
- **One write query:** an entry can be an object - `{"name": "submit_order", "allow_writes": true}` - giving write access to that one query only, still within `QUERYAPIGATE_ALLOW_WRITES`. See Allow a saved query to write data.
- **Every saved query, but never raw SQL:** `"queries": "*"`.
- **Tighten it further:** a partner key is a good candidate for a rate limit, an IP allowlist and an expiry date - all fields on the same key.

Next steps

- Give several partners the same access with a role.
- Hand the partner a Postman collection - put the query in a collection first.
- Verify who can reach what.